



Data Manipulation & Integrity

Mastering essential commands for robust database management.

Mastering how to **modify data safely** and **link tables together** is crucial for any developer or database professional. This session will equip you with essential SQL skills and best practices.

1

DML Commands

(INSERT, UPDATE, DELETE)

2

Database Constraints

Ensuring data quality

3

Table Relationships

Connecting your data

4

Referential Integrity

Maintaining consistency

Inserting New Data: The **INSERT** Command

The **INSERT INTO** statement is your gateway to populating tables with new records. Understanding its proper use is fundamental to database management.

Key Concepts:

- Adds new rows to a table.
- You must specify the target table, the columns you're populating, and their corresponding values.
- Always list column names explicitly to prevent errors if the table structure changes in the future.

Example:

```
INSERT INTO Students (StudentID, FirstName,  
LastName, EnrollmentDate) VALUES  
(101, 'John', 'Doe', '2026-02-11');
```

 Best practice: Always match column order with value order.

Modifying Existing Data: The **UPDATE** Command

The **UPDATE** statement allows you to change information in existing rows. This is where precision becomes paramount.

Understanding UPDATE:

- The **SET** clause defines which columns to modify and what their new values will be.
- The **WHERE** clause is **CRUCIAL**: it filters and identifies the specific rows to be updated.

Critical Warning!

If you omit the **WHERE** clause, **ALL rows** in the table will be updated!

Example:

```
UPDATE Students SET LastName = 'Johnson'  
WHERE StudentID = 101;
```

Removing Data: The **DELETE** Command

When data is no longer needed, the **DELETE FROM** statement is used to remove rows. Just like UPDATE, the **WHERE** clause is key.

DELETE in Action:

- Removes one or more rows from a specified table.
- The **WHERE** clause precisely targets which rows to eliminate.

Grave Warning!

Forgetting the **WHERE** clause will result in **ALL rows being deleted**, leaving your table empty!

Example:

```
DELETE FROM Students  
WHERE StudentID = 101;
```

📌 Always double-check your WHERE clause before executing DELETE.

Modifying Table Structures: The **ALTER TABLE** Command

The **ALTER TABLE** command is essential for modifying the structure of an existing table after it has been created. This includes adding, modifying, or dropping columns, among other changes.



ADD COLUMN

Adds a new column to an existing table. You specify the new column's name and its data type.

```
ALTER TABLE Students  
ADD Email VARCHAR(5);
```



ALTER COLUMN

Modifies the definition of an existing column, such as changing its data type or nullability.

```
ALTER TABLE Students  
ALTER COLUMN Email  
VARCHAR(150) NOT NULL;
```



DROP COLUMN

Removes an existing column from a table. This action is irreversible, so proceed with caution!

```
ALTER TABLE Students  
DROP COLUMN Email;
```

Removing Entire Structures: The **DROP** Commands

Beyond individual rows and columns, SQL allows you to permanently remove entire tables and databases. These powerful commands are irreversible and demand extreme caution.

DROP TABLE

The `DROP TABLE` statement permanently deletes an existing table, along with all its data, indexes, and constraints.

- All data and the table definition are immediately lost.
- This action cannot be undone without a prior database backup.

```
DROP TABLE Students;
```

 **Best Practice:** Always verify the table name and backup your database before execution.

DROP DATABASE

The `DROP DATABASE` statement permanently deletes an entire database, including all its tables, data, views, stored procedures, and other objects.

- Deletes everything associated with the specified database.
- Requires appropriate database administration permissions.
- Ensure no users are connected to the database before dropping.

```
DROP DATABASE UniversityDB;
```

CRITICAL WARNING: Never execute `DROP DATABASE` on a production system without explicit approval and a verified, recent backup!

Safeguarding Your Data: Database Constraints

Constraints are rules that enforce data integrity and accuracy within your database. They are fundamental for maintaining a reliable system.

PRIMARY KEY

A unique identifier for each record. It cannot contain duplicate values or NULLs, ensuring every row is distinct.

FOREIGN KEY

Establishes a link to the Primary Key in another table, defining and enforcing relationships between tables.

NOT NULL

Ensures that a column cannot be left empty. Essential for data that must always be present, like a student's last name.

UNIQUE

Guarantees that all values in a specific column are different. Useful for ensuring no two records share the same email address.

Putting Constraints Together: **CREATE TABLE**

Combining **PRIMARY KEY**, **UNIQUE**, and **NOT NULL** constraints in your table definition ensures robust data integrity right from the start.

Here's an example of creating a `Customers` table with these fundamental rules applied:

```
CREATE TABLE Customers (  
  CustomerID INT PRIMARY KEY,  
  Email VARCHAR(255) UNIQUE NOT NULL,  
  FirstName VARCHAR(100) NOT NULL,  
  LastName VARCHAR(100),  
  RegistrationDate DATE NOT NULL  
);
```



CustomerID: Ensures each customer has a unique identifier and is never null.

Email: Guarantees every customer has a unique and non-empty email.

FirstName, RegistrationDate: Must always have a value.

Adding Constraints to Existing Tables: The **ALTER TABLE** Command

Sometimes you need to apply rules to tables that already exist. The **ALTER TABLE** statement allows you to modify a table's structure, including adding new constraints.

Why ALTER TABLE?

- Apply constraints to existing data.
- Update table definitions as requirements evolve.
- Refine data integrity rules without recreating the table.

Examples of Adding Constraints:

```
-- Add a PRIMARY KEY
ALTER TABLE Customers
ADD CONSTRAINT PK_CustomerID PRIMARY KEY
(CustomerID);
```

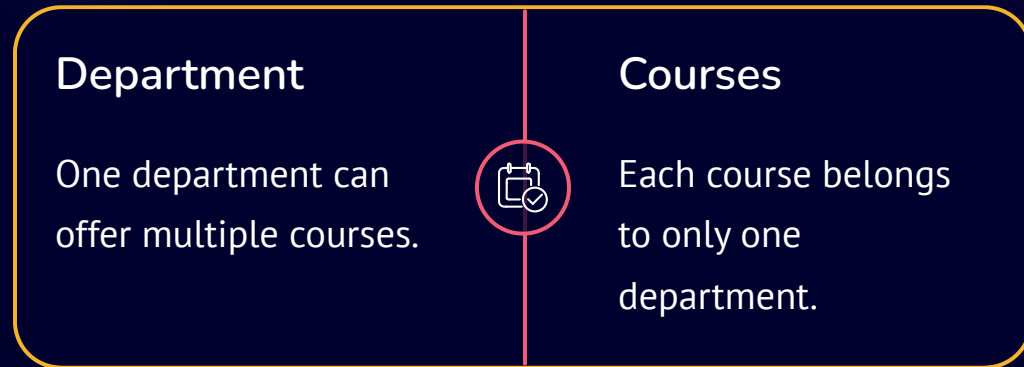
```
-- Add a UNIQUE constraint
ALTER TABLE Customers
ADD CONSTRAINT UQ_Email UNIQUE (Email);
```

```
-- Add a NOT NULL constraint
ALTER TABLE Customers
ALTER COLUMN FirstName VARCHAR(100) NOT
NULL;
```

Understanding Table Relationships

Databases aren't just collections of isolated tables; they are networks of interconnected information. Defining these relationships is key to structured data.

One-to-Many Relationship



Many-to-Many Relationship



📌 This type requires a "Junction Table" (e.g., Enrollments) to connect them.

Creating Related Tables with Foreign Keys

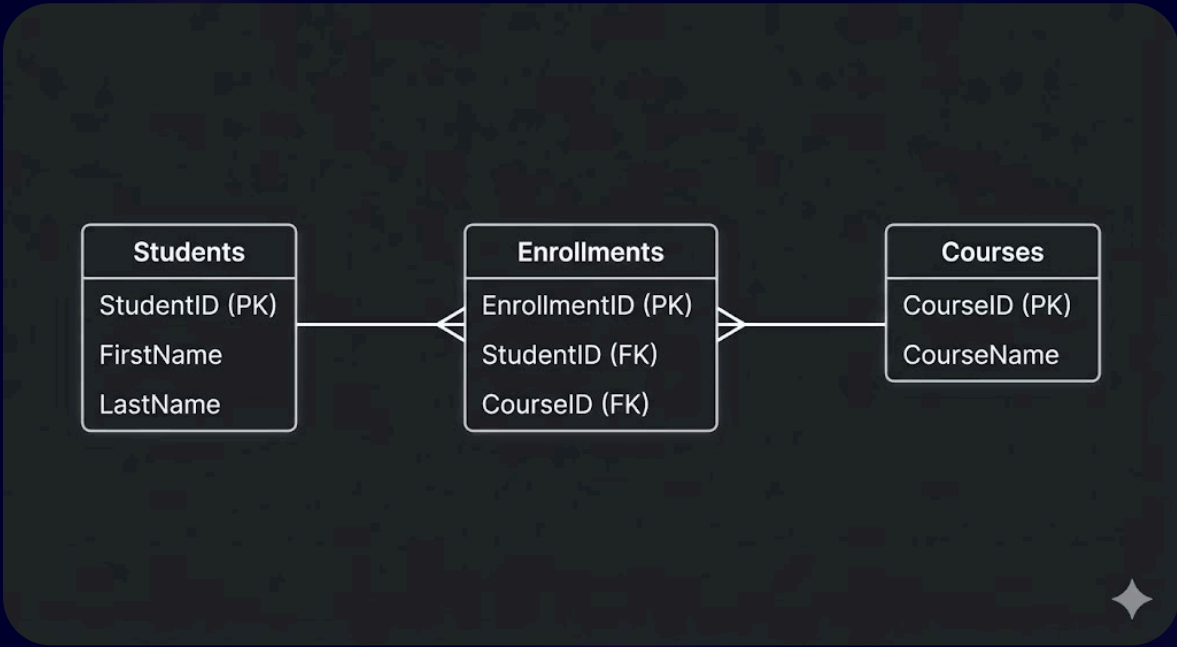
Foreign keys are crucial for establishing and enforcing links between tables. They ensure that relationships, like a department offering courses, are maintained consistently.

```
CREATE TABLE Departments (  
    DepartmentID INT PRIMARY KEY,  
    DepartmentName VARCHAR(100) NOT NULL UNIQUE  
);  
  
CREATE TABLE Courses (  
    CourseID INT PRIMARY KEY,  
    CourseName VARCHAR(255) NOT NULL,  
    DepartmentID INT,  
    FOREIGN KEY (DepartmentID) REFERENCES Departments(DepartmentID)  
);
```

📌 The `DepartmentID` in the **Courses** table acts as a **FOREIGN KEY**, referencing the **PRIMARY KEY** of the **Departments** table. This prevents courses from being assigned to non-existent departments.

Creating Related Tables with Foreign Keys

To implement the many-to-many relationship between students and courses, we need three tables: one for **Students**, one for **Courses**, and a **Junction Table** to link them. The junction table uses Foreign Keys to reference the Primary Keys of the other two tables.



This diagram illustrates how the Enrollments table acts as a crucial junction, linking Students and Courses to form a many-to-many relationship.

Students Table

Stores unique student information.

```
CREATE TABLE Students (  
  StudentID INT PRIMARY KEY,  
  FirstName VARCHAR(100) NOT NULL,  
  LastName VARCHAR(100) NOT NULL  
);
```

Enrollments Table

The crucial junction table linking students to courses.

```
CREATE TABLE Enrollments (  
  EnrollmentID INT PRIMARY KEY,  
  StudentID INT,  
  CourseID INT,  
  FOREIGN KEY (StudentID) REFERENCES  
  Students(StudentID),  
  FOREIGN KEY (CourseID) REFERENCES  
  Courses(CourseID)  
);
```

❏ The **Enrollments** table creates a **many-to-many relationship** by storing pairs of **StudentID** and **CourseID**, each acting as a Foreign Key to their respective tables.

Ensuring Consistency: Referential Integrity

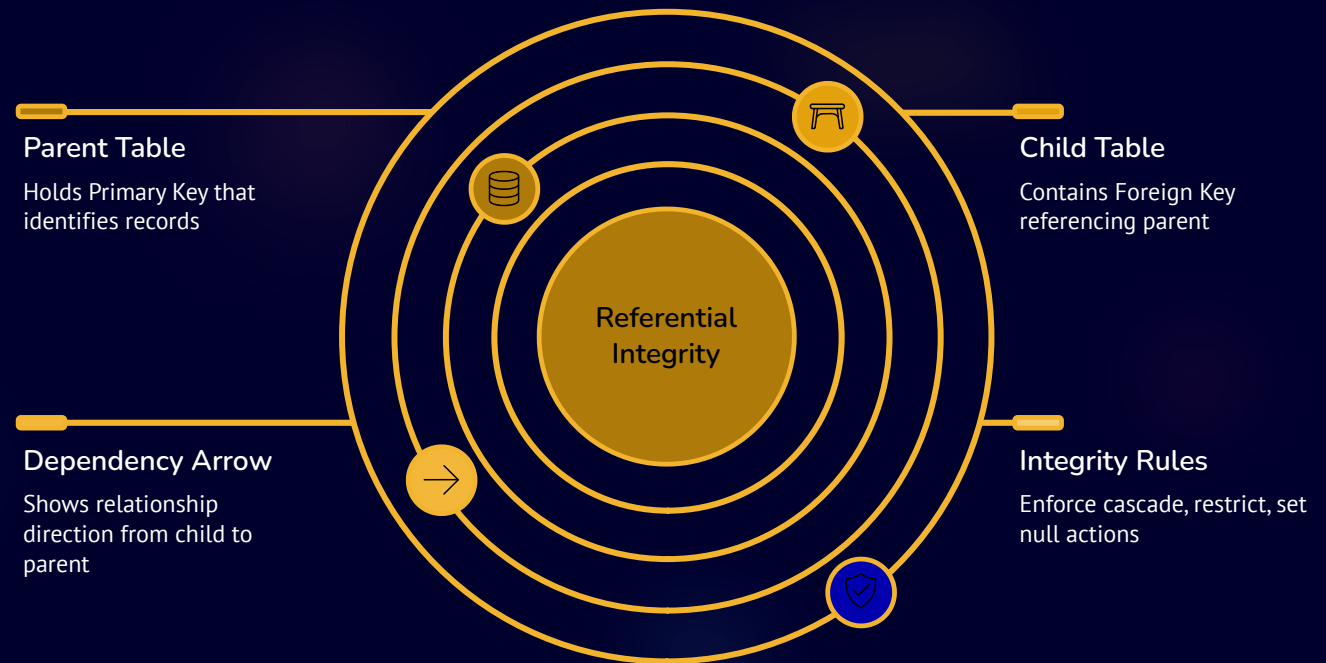
Referential Integrity is a critical database concept that ensures relationships between tables remain valid and consistent over time, preventing data corruption.

Core Definition:

- Keeps data consistent between related tables.
- Enforces the rule: you cannot refer to data that does not exist in the referenced table.

Practical Implications:

- You cannot enroll a StudentID that isn't present in the Students table.
- You generally cannot delete a Course if students are actively enrolled, preventing "orphaned" records (unless "ON DELETE CASCADE" rules are explicitly set).



This mechanism prevents the creation of dangling references, ensuring your data relationships are always sound and meaningful.